

Descriere algoritmi de implementat:

Proiectul definește 3 structuri:

- **Vec3** – punct 3D (x, y, z)
- **Edge** – muchie între 2 varfuri (a, b), a și b sunt indici în vectorul de varfuri
- **Framebuffer** – buffer care scrie pixelii, înainte de a fi afișați pe ecran

1. Rotirea unui punct în jurul axei Y:

Scop: Rotirea unui punct/puncte din spațiul 3D în jurul axei verticale (Y) pentru ca modelul să pară animat.

O funcție `rotate_y(Vec3 v, float angle)` folosește formula clasică de rotație 3D:

$$\begin{aligned}x' &= x \cos \theta + z \sin \theta \\y' &= y \\z' &= -x \sin \theta + z \cos \theta\end{aligned}$$

Pentru fiecare punct, se calculează coordonatele noi folosind cosinus și sinus. Formula se aplică pe coordonatele X și Z, deci coordonata Y rămâne neschimbată (putem extinde algoritmul pentru animații mai complexe)

Complexitate: $O(1)$, per punct, foarte rapid.

2. Proiecția 3D-2D (perspectiva simplă):

Scop: Transformarea coordonatelor 3D într-un sistem de coordonate 2D pentru a fi desenate pe ecran, creând efectul de **perspectivă**, astfel încât obiectele mai îndepărtate să pară mai mici.

Funcția `project()` convertește un punct 3D în coordonate de ecran:

$$factor = \frac{scale}{z + z_{camera}}$$

(factorul de scalare depinde de distanța Z față de camera)

$$\begin{aligned}x_{screen} &= x \cdot factor + width/2 \\y_{screen} &= -y \cdot factor + height/2\end{aligned}$$

- Se ajustează Z-ul cu distanța camerei (z_{off}) pentru a evita distanțele negative.
- Se aplică formula perspectivei simple: obiectele îndepărtate sunt scalate mai mic.
- Se centerizează pe ecran adăugând jumătate din lățimea și înălțimea ferestrei.
- Se inversează Y-ul (-y) deoarece coordonata verticală a ferestrei crește de sus în jos, dar în spațiul 3D Y crește de jos în sus.

Complexitate: $O(1)$, per punct, foarte rapid.

3. Algoritmul de generare wireframe din fețe .obj:

Scop: Dintr-un model .obj, extrage vârfurile și fețele și le transformă în **muchii** pentru a fi desenate cu Bresenham.

Un fisier de tip .obj (la modul cel mai simplu) contine proprietatile unui model 3D prin urmatoarea sintaxa:

```
o Cube

#Lista de puncte 3D (vertices), cu coordonatele x, y, z
v 1.000000 1.000000 -1.000000
v 1.000000 -1.000000 -1.000000
v 1.000000 1.000000 1.000000
v 1.000000 -1.000000 1.000000
v -1.000000 1.000000 -1.000000
v -1.000000 -1.000000 -1.000000
v -1.000000 1.000000 1.000000
v -1.000000 -1.000000 1.000000

#Fetele corpului geometric, definite prin liste de varfuri
f 1 5 7 3
f 4 3 7 8
f 8 7 5 6
f 6 2 4 8
f 2 1 3 4
f 6 5 1 2
```

Funcția **load_obj()** citește un fișier de tip .obj minimal:

- Linie **v x y z** : adaugă un vârf în vectorul **verts**.
- Linie **f i j k...**: genereaza muchii între varfurile fetei
 - Se genereaza muchii $i \rightarrow j, j \rightarrow k, \dots$ și ultima $k \rightarrow i$.
 - Deci, poligonul este descompus in muchii pentru randarea wireframe

Algoritmul:

- Citește fiecare linie.
- Dacă e **vârf**, parsează coordonatele X,Y,Z și adaugă în array.
- Dacă e **față**, parsează indicii varfurilor ptr fete poligonale.
- Se genereaza muchiile fetei:
 - fiecare muchie = vârff[i] $\rightarrow$ vârff[i+1]
 - ultima muchie = vârff[n-1] $\rightarrow$ vârff[0] (închide poligonul).
- Toate muchiile sunt stocate într-un vector dinamic **edges[]** pentru desenarea wireframe-ului.

Complexitate: $O(n + m)$ pentru n vârfuri și m fețe, deoarece fiecare față se parcurge o singură dată.

4. Algoritmul Bresenham pentru linii:

Scop: Desenarea unei linii între două puncte pe un grid discret (pixeli). Evită folosirea floating point și permite linie continuă fără spații vizibile.

- Se calculează diferențele absolute **dx** = $|x1 - x0|$ și **dy** = $|y1 - y0|$.
- Se determină direcțiile **sx** și **sy** (+1 sau -1) în funcție de poziția punctelor.

- Se inițializează **err = dx + dy** (sumă de erori).
- Într-un loop:
 - Se colorează pixelul curent.
 - Se verifică dacă am ajuns la destinație.
 - Se ajustează err și se mută x sau y după cum e necesar pentru a menține linia continuă.

Complexitate: $O(n)$ unde n este lungimea liniei în pixeli.

5. Loop-ul principal:

Scop: Actualizarea și afișarea continuă a wireframe-ului pe ecran (~60 FPS) și rotirea modelului + gestionarea bufferului software + texture SDL

- **Event polling:** verifică dacă fereastra s-a închis (**SDL_QUIT**).
- **Redimensionare fereastră:**
 - Dacă dimensiunea ferestrei s-a schimbat:
 - Se recrează textura SDL cu dimensiunea nouă.
 - Se realocă buffer-ul software.
- **Clearing buffer:** Se scrie 0xFF000000 (negru opac) în fiecare pixel.
- **Rotație + proiecție + desenare:**
 - Pentru fiecare muchie:
 - Rotează vârfurile folosind **rotate_y**.
 - Proiectează punctele 3D → 2D folosind **project**.
 - Desenează linia cu Bresenham în buffer.
- **Actualizare SDL:**
 - Copiază buffer-ul software în textura SDL (**SDL_UpdateTexture**).
 - Desenează textura în renderer (**SDL_RenderCopy**).
 - Prezintă frame-ul (**SDL_RenderPresent**).
- **Increment unghi:** pentru animație continuă.
- **Delay:** pauză ~16ms (~60 FPS).

Observatie: Calculele grafice sunt realizate de catre software, SDL doar afiseaza buffer-ul pe ecran

Vitis foloseste limbajul C pentru implementarea codului, aceasta suportand librariile **stdlib.h** (alocare memorie), **math.h** (functii matematice) si **stdio.h** (citire fisiere).

Echivalentul codului in C simplu foloseste libraria SDL pentru a gestiona fereastra si bufferul grafic (mai precis **SDL_RenderClear**, **SDL_RenderPresent** si **SDL_GetWindowSize**), insa in absenta acestuia in Vitis, va trebui sa afisam manual pe un monitor prin HDMI modelul 3D:

1. Video Timing Generator (VTG):

Scop: Să genereze sincronizările HDMI:

- HSYNC
- VSYNC
- DE (Data Enable)
- Pixel Clock

Fara acestea, monitorul rămâne albastru/negru.

Un VTG este un **automat finit** bazat pe contoare:

- $h_counter$ → poziția pixelului pe linie
- $v_counter$ → linia curentă

Parametri (ex. 800x600@60Hz)

- Horizontal:
 - active pixels
 - front porch
 - sync pulse
 - back porch
- Vertical:
 - active lines
 - front porch
 - sync pulse
 - back porch

Algoritm (conceptual)

la fiecare pixel_clock:
 $h_counter++$
dacă $h_counter == H_TOTAL$:
 $h_counter = 0$
 $v_counter++$
dacă $v_counter == V_TOTAL$:
 $v_counter = 0$

Semnale generate:

HSYNC = ($h_counter$ în zona de sync)

VSNC = ($v_counter$ în zona de sync)

DE = ($h_counter < H_ACTIVE \ \&\& \ v_counter < V_ACTIVE$)

2. Algoritmul de scan-out framebuffer:

Scop: Citirea pixelilor din RAM EXACT în ordinea cerută de HDMI.

Principiu: HDMI NU știe ce e „linie Bresenham”. El cere: „Dă-mi pixelul (x,y) ACUM”

Algoritm:

dacă $DE == 1$:
 $address = y * WIDTH + x$
 $pixel = framebuffer[address]$
altfel:
 $pixel = 0 \text{ (negru)}$

Unde:

- $x = h_counter$

- $y = v_counter$

3. Algoritmul DMA (când framebufferul e în DDR):

Scop: Să muți datele rapid din DDR → logică video.

În Vitis:

- AXI VDMA
- AXI Stream

Flux:

DDR framebuffer
 ↓ (*DMA burst*)
Line Buffer
 ↓
HDMI pixel stream

Algoritm DMA

1. Inițializează adresa de start framebuffer
2. Pentru fiecare linie:
 - citește WIDTH pixeli
 - livrează la pixel clock
3. Repetă pentru fiecare frame

DMA rulează independent de CPU. CPU doar schimbă pointerul de framebuffer (double buffering).

4. Algoritmul de double buffering (esențial)

Scop: Evitarea fenomenului de **tearing** (jumătate cadru vechi, jumătate nou).

Structură

FB0 ← se afișează

FB1 ← CPU desenează (Bresenham, etc)

Algoritm:

la VSYNC:
swap(FB0, FB1)

CPU:

- scrie în bufferul invizibil

Video:

- citește din bufferul activ

Echivalentul lui `SDL_RenderPresent()`, dar în hardware

5. Algoritmul HDMI TMDS Encoding

Asta nu se face manual.

TMDS:

- 8-bit → 10-bit encoding
- minimizare tranziții
- DC balancing

În Vitis:

- **HDMI TX Subsystem**
- sau IP dedicat

Proiectul livrează doar **pixeli + sync**, IP-ul face encoding-ul.

Echivalente SDL vs hardware:

SDL	HDMI hardware
SDL_Window	Video Timing Generator
SDL_Texture	Framebuffer DDR
SDL_UpdateTexture	DMA
SDL_RenderCopy	Scan-out
SDL_RenderPresent	VSYNC swap

Lista FINALĂ de algoritmi pentru echivalentul proiectului în hardware:

Algoritmi **VIDEO**:

1. Video Timing Generator (HSYNC / VSYNC)
2. Pixel Scan-out (x,y → framebuffer)
3. DMA Burst Reader
4. Double Buffering
5. TMDS Encoding

Algoritmi **GRAFICI**:

6. Parsarea fisierului OBJ
7. Rotatie 3D
8. Proiecție 3D → 2D

9. Bresenham (linie)

10. Clear framebuffer

11. Scriere in framebuffer